



SEN381 ASSIGNMENT 2

Research project

Armand Johannes Vermeulen 602636
Ditheto Lesiba Phaho 601133
Riegardt Christopher Weich 602547

Task 1

4.1 Establishing the Design Problem

Coupling and Cohesion

Coupling and cohesion are the two foundational measures of internal design quality first formalised by Stevens, Myers and Constantine (1974) in their structured-design method. Coupling describes the degree of interdependence between separate modules or classes — how much one component must know about, or depend on, the internal detail of another. Cohesion describes how strongly the responsibilities within a single module or class relate to one another (Stevens, Myers and Constantine, 1974). The two measures work together: a design goal of low coupling and high cohesion produces modules that can be understood, tested and changed in isolation, because a class with one clear responsibility rarely needs to change for reasons unrelated to that responsibility, and a class with few dependencies on other classes can be modified without a ripple of changes elsewhere (Sommerville, 2016).

These properties matter specifically once a system has multiple classes or components that must evolve together, which is the point CivicConnect is now approaching. If a single class both accepts a service request and decides how every interested party (a notification, an audit log, a future escalation rule) should react to it, that class is doing more than one job (low cohesion) and is directly dependent on the implementation detail of every one of those reactions (high coupling). Every time a new reaction is required, that shared class must be reopened and modified — the opposite of a controlled, low-risk change.

Relevant SOLID Principles

Of Martin's (2003) five SOLID principles, three are directly relevant to the design problems identified below:

- **Single Responsibility Principle (SRP)** — a class should have only one reason to change (Martin, 2003). This is the principle most directly violated if request-handling logic also contains notification, audit and future-escalation logic.
- **Open/Closed Principle (OCP)** — software entities should be open for extension but closed for modification (Martin, 2003). CivicConnect's category taxonomy (Scope Baseline, Section 3.1) and its deferred future-scope items (SLA escalation, advanced reporting) mean new categories and new reactions to events are expected over the project's life; the design should accommodate that growth without repeated edits to already-tested code.
- **Dependency Inversion Principle (DIP)** — high-level modules should not depend on low-level modules; both should depend on abstractions (Martin, 2003). This underpins both pattern comparisons below, since Observer, Mediator, Strategy and Factory Method are all, in different ways, mechanisms for depending on an abstraction rather than a concrete implementation.

Two Genuine CivicConnect Design Problems

Described here at the problem level, before any pattern is named, as required by the assignment brief:

Design Problem 1 — Reacting to a service-request status change. Several parts of CivicConnect must react whenever a request's status changes: the requester must receive feedback (FR-006), the change must be recorded for auditability (NFR-020), and management must be able to see major activity (FR-014). Further reactions are expected later (deferred SLA escalation, Section 3.3 of the PED). If the class responsible for changing a request's status also directly calls the notification code, the audit-log code, and — eventually — the escalation code, that class becomes coupled to every consumer of the event

and must be edited every time a new consumer is added. The problem is: how should CivicConnect notify multiple, independent, and growing sets of interested parties about a status change without the core request-handling logic knowing about each of them individually?

Design Problem 2 — Applying category-specific validation and behaviour. CivicConnect requires a controlled, fixed category taxonomy (Scope Baseline, Section 3.1) and mandates validation and clear feedback at submission (NFR-021). Different categories plausibly require different validation rules in practice (for example, a security-concern request may need different mandatory fields than a lost-property request). A naive implementation would place a growing chain of `if category == "X"` conditionals inside the submission-handling code. This directly violates OCP, since every new category requires editing that shared block, and it lowers cohesion, since one class ends up holding the validation rules for every category rather than one clear responsibility. The problem is: how should category-specific validation behaviour be represented so that adding a new category does not require modifying existing, already-tested submission logic?

4.2 Researching Alternatives

Design Problem 1 — Notifying Interested Parties of a Status Change

Candidate A: Observer Pattern The Observer pattern defines a one-to-many dependency so that when one object (the *subject* — here, the service request) changes state, all registered *observers* are notified automatically, without the subject needing to know their concrete types (Gamma et al., 1994). A notification handler, an audit-log handler, and a future escalation handler would each register as observers; the request object simply announces "status changed" once.

Effect on design quality: this directly reduces coupling — the request class depends only on a generic observer interface, not on notification or audit implementation detail — and improves cohesion, since the request class's only responsibility becomes managing its own state and announcing changes to it. Extensibility improves because a new observer (e.g. the deferred escalation feature) can be added without modifying the request class at all, satisfying OCP. Testability improves because each observer can be unit-tested independently, with the subject's behaviour verified through a mock observer.

Risks: the update ordering between multiple observers is not guaranteed and, if used carelessly, chains of observers reacting to each other's changes can become difficult to trace (Gamma et al., 1994). For a system as small as CivicConnect's current scope, an unnecessarily generic publish/subscribe mechanism could also be over-engineering if only one or two observers ever exist.

Candidate B: Mediator Pattern The Mediator pattern centralises interaction logic in a single mediator object, so that request-related components communicate through the mediator rather than referencing each other directly (Gamma et al., 1994). Under this design, the request class would inform a "request status mediator," which would then be solely responsible for knowing how to invoke notification, audit and escalation logic.

Effect on design quality: Mediator also removes direct coupling between the request class and each downstream concern, and centralises what would otherwise be scattered interaction logic, which can improve maintainability when many components interact in complex, many-to-many ways. However, cohesion at the system level can suffer if the mediator itself grows into a large class that has to know about every interested party's calling convention — the coupling is not removed, it is concentrated in one place (Gamma et al., 1994).

Comparison and when each is appropriate: Observer is the more proportionate choice when the relationship is genuinely one-to-many and the reacting parties do not need to coordinate with each other, which matches CivicConnect's case (notification, audit, and future escalation do not need to know about one another). Mediator is more appropriate when several components need to interact with each other in a coordinated way, which is not yet a problem CivicConnect has. A simpler alternative — a direct, hard-coded sequence of method calls — remains preferable only while the number of reactions is fixed at one and no extensibility is anticipated; since CivicConnect already has three current reactions and at least one confirmed future one, that simpler approach is not appropriate here.

Design Problem 2 — Category-Specific Validation Behaviour

Candidate A: Strategy Pattern The Strategy pattern defines a family of interchangeable algorithms, encapsulates each one, and makes them substitutable behind a common interface, so the calling code can select a behaviour without knowing its implementation (Gamma et al., 1994). Each request category would have its own validation-strategy class implementing a shared RequestValidator interface; the submission handler simply asks for "the validator for this category" and calls it.

Effect on design quality: this satisfies OCP directly — a new category means adding a new strategy class, not editing existing ones — and improves cohesion, since each strategy class has exactly one responsibility (validating one category of request). It also improves testability, since each category's validation rules can be tested in isolation.

Risks: if categories rarely differ in their validation logic, Strategy introduces more classes than the problem justifies (Gamma et al., 1994) — a form of over-engineering for a small, fixed set of simple categories.

Candidate B: Factory Method Pattern The Factory Method pattern defines an interface for creating an object but lets subclasses decide which concrete class to instantiate (Gamma et al., 1994). Applied here, a factory would be responsible for constructing the correct validator (or request-handling object) for a given category, deferring the decision of *which* concrete class to create to category-specific factory subclasses.

Effect on design quality: Factory Method is well suited to the problem of *object creation* varying by category, and also supports OCP by allowing new creation logic to be added via a new factory subclass. However, it solves a narrower problem than Design Problem 2 actually presents: the difficulty here is not primarily "which object should be created," but "which validation behaviour should run." Using Factory Method alone would still require the *created* object to contain the varying behaviour — in practice, a factory is often paired with Strategy rather than used as a substitute for it (Gamma et al., 1994).

Comparison and when each is appropriate: Strategy is the more direct fit for Design Problem 2, because the problem is behavioural (which validation rules apply), not primarily about object instantiation. Factory Method becomes more clearly justified if CivicConnect later needs to construct entire category-specific request objects with different fields or structure, not just different validation rules — a scenario the current requirements do not yet demonstrate. A simpler conditional (if/switch on category) remains adequate only while the category list is small and stable; given that the category list is explicitly described as a controlled but extensible taxonomy (Scope Baseline, Section 3.1), that simpler approach is expected to become a maintenance liability as categories are added.

4.3 Recommendations Feeding M2

Design Problem 1 (status-change reactions):

The research indicates the **Observer pattern** is the more proportionate recommendation for CivicConnect. The relationship between a request's status change and its interested parties (notification, audit, future escalation) is a one-to-many relationship with no coordination needed between observers, which is exactly the case Observer is designed for, and it satisfies OCP for the escalation feature already identified as future scope. This recommendation should inform the M2 architecture and design decisions covering the request-status workflow, and should be recorded as an ADR once the team commits to it, since it is an architecturally significant decision under the Master Project Brief's ADR standard.

Design Problem 2 (category-specific validation):

The research indicates the **Strategy pattern** is the more proportionate recommendation, since the problem is behavioural rather than creational. This recommendation should inform the M2 design decisions for request submission and validation (feeding FR-001, FR-002 and NFR-021), and should likewise be recorded as an ADR, with Factory Method noted as a candidate to revisit only if category-specific object structure (not just validation behaviour) becomes a requirement.

As required by the assignment brief, both recommendations are treated as evidence for M2, not as pre-made M2 commitments — the team will still record its final design decision, rationale and application evidence in the PED/ADR/RTM once the corresponding M2 decision is formally baselined.

Task 2

To determine how CivicConnect should preserve data correctness, we require research to understand the situation and explore the topic further before implementing. The research will be used to guide the decisions made in Milestone 2.

5.1 Choose one meaningful business operation.

The chosen operation is service request creation.

Example: A citizen submits a civic issue; issues can vary from potholes to water leaks. The operations takes the request information and stores its data such as name, description, and initial entry history.

This operation is critical because the system needs accurate and complete data to function as intended. Partial data could result in a request existing without a audit history, duplicate request for repeated submissions and incomplete data that can not be tracked because of data integrity concerns.

This operations requires mechanisms across multiple updates and services to ensure data correctness is guaranteed.

5.2 Research the engineering choices.

A. Transaction and atomicity requirements

A transaction in a system represents a group of operations that are treated as one indivisible unit of work. According to (Gray & Reuter, 1993), atomicity ensures that all operations succeed or if anyone fails the entire transaction is not applied. This prevents partially complete records from infecting the database or system with dirty data.

For CivicConnect the operations to ensure the transaction is atomic follows:

1. Create request.
2. Record initial date and assign open status.
3. Save all relative metadata that the system will use.
4. Commit the transaction once all data criteria are met.

If any of the steps named above fail the record will not be saved. This is done with a roll back to ensure consistency.

B. Validation and business rules

Research by (OWASP Foundation,2024) supports that data validations should be enforced across multiple layers rather than on single point of validation.

Layer	Responsibly	Example
UI/Client	Ensure user input is validated before further processing.	Non null values, non-negative values, valid date-time, phone number and email formats.
Application/Service	Business rules and logic	Prevent duplicate submissions, validate correct categories.
Database	Integrity constraints	Unique request IDs, mandatory fields and referential integrity.

Consequences of relying on a singular point of validation. UI only validation can be bypassed by direct requests outside of the user interface. Database only could lead to poor user experience and limited business rules. Service only could lead to structural data integrity inconsistencies.

C. Consistency, concurrency and integrity

CivicConnect is a multi-user system. Meaning multiple users will access and alter the same data, this could lead to inconsistencies in data states, concurrency control problems and bad database integrity, causing errors in the system.

Mechanisms to ensure consistency, concurrency and integrity:

- For consistency every completed transaction should leave the database in a valid state.
- For concurrency the system should prevent updating the same record while multiple users are interacting with the same record.
- For integrity the system must force relationships and required data to remain in a valid state.

Research by (Kistijantoro A, Morgan G, Shrivastava S and Little M 2008) shows controlled transactions reduce the risk of anomalies forming in the system, such as lost updates, inconsistent records and data integrity in collaborative systems.

D. Caching: correctness vs staleness

Caching is a useful tool to increase read performances and put less load on the database but introduces the risk of stale data. CivicConnect receives frequent updates to data, particularly request statuses.

Cached data	Recommendation
Creating Requests	Change frequently: Should not be cached
Status updates	Change frequently: Should not be cached
Read only dashboards	Should be cached:

According to research from (Wang L and Stoller S (2006)): Correctness must take priority over performance for transactional operations. Making caching for creating request and updating statuses inappropriate for our case.

E. Comparison of implementation approaches

Criteria	Database-centred	Service-layer responsibility
Business/Logic rules	Limited	Strong
Data integrity	Strong	Strong
Maintainability	Medium	High
Testability	Low	High
Separation of concerns	Low	High

Database centred approach: If we rely on a database centred approach the validation and transactional logic is implemented in the database. This leads to Strong integrity but leaves us with Low testability and tightly coupling business rules within the presentation layers. (Kistijantoro A, et al, 2008)

Service Layer responsibility approach: Using the service layer to handle transactions and business rules. This solves the problem of low separation of concerns and testability problems the database-centred approach introduces. While the database can still be used to enforce structural integrity through constraints. (Kistijantoro A, et al, 2008)

The Service Layer responsibility approach is superior for a system wide efficiency and is optimal for good software engineering practices.

5.3 Recommendation feeding the project.

After considering the research, and comparing it to our current project requirements and engineering choices. We can conclude the recommended structure is that the Service Request Creation operation should preserve correctness through a layered responsibility approach:

- UI: Validates user inputs against processing format and provides immediate user feedback
- Application/Service: Control the transactions and enforce business rules
- Database: Enforce data integrity constraints and guarantee atomic preservation

This recommendation informs Milestone 2 artefacts

Milestone 2 Artefacts	Decision informed
Data persistence	Transaction boundaries
Architecture Decision Record (ADR)	Validations responsibility allocation
Risk register	Preventing partial or duplicate records/requests
Data model	Integrity constraints and inter-table relationships

This approach provides high data integrity without concentrating all the responsibilities into a single layer. Which align with research of well-established engineering practices (OWASP Foundation,2024).

Task 3

This research sections discusses how CivicConnect modules should communicate while avoiding unnecessary coupling. The main goal is to identify the integration problems and finding the appropriate mechanisms before architectural decisions are baselined.

6.1 Identify an integration problem

Selected integration problem:

Request module => Notification module

How will the request module communicate with the notification module when a user creates a request? The work flow will be: User creates/updates a service request. Then the notification module informs the user that the request has been received or the status has been updated.

Producer and consumer

Role	Component
Producer (User request)	Request Management Module
Consumer (System response)	Notification Module

Information exchanged

- Request ID
- User ID
- Request Status
- Timestamp
- Notification message

Failure consequences

If the integrations fails, the following consequences could follow:

- User won't receive notification of their interaction.
- Multiple notification for the same request could be generated.
- Request management module and notification module could be tightly coupled leading to difficulty updating the system without leading to cascading problems.

This integration problem requires a explicit engineering decision because communication between modules is crucial for the system to function properly, to ensure maintainability and handle failures more effectively.

6.2 Compare integration approaches

In-Process approach:

Modules communicate directly through direct methods like interface calls within the same application Thomas, R. (2000).

Advantages for this approach:

- Low latency
- Simple
- Easy unit testing
- No network overhead

Limitations for this approach:

- Dependencies between modules are tighter coupled
- Less flexibility when changing modules
- Caller is affected immediately by callee failures

HTTP/REST API Approach:

Modules communicate through RESTful requests across defined interfaces (Thomas, R. 2000).

REST research identifies several important design principles:

- The resources should represent business entities.
- HTTP methods should express operations consistently
- Communication should remain stateless.
- Standard status code practices should be enforced for consistency and separation between failure and success.

Advantages for this approach:

- Can separate modules more effectively, to encourage loose coupling.
- Modules and components can be updated and changed easier without affecting the entire system
- Better security boundaries due to data crossing over organization boundaries such as firewalls (Thomas, R. 2000).

Limitations for this approach:

- Network latency
- Error handling becomes more complex
- Monitoring and deployment overhead

Factor	In-Process interface	HTTP/REST API
Coupling	Medium	Low
Performance	High	Medium
Failure behaviour	Shared process	Network Failures possible
Testability	High	High (but more complex)
Deployment	Single application	Distributed components
Security boundary	Internal only	Explicit
Version compatibility	Limited	Wide

Cost of distribution

Research on distributed system warns against unnecessary distribution because it leads to increased latency, operational complexity, and failure scenarios. A network boundary should only be introduced if a component is deployed outside of the system and require access to the rest of the system. Therefore, HTTP/REST API shouldn't be the automatic choice because it's the most used.

6.3 Recommendation feeding the project

For CivicConnect choosing a in-process interface between the request manager and notification modules is the best option. No need to add additional complexity if the modules will live on the same server.

Justification

- Both modules process on the same server/business application.
- There is no found evidence to distribute them a across a network.
- Better latency, lower complexity, easier testing and maintaining separation between defined contracts.
- REST should be used for external deployed integrations rather than internal communication(Kistijantoro A, et al 2006).

M2 project artefacts informed

Project Artefact	Decision Informed
Architecture diagram	Internal module communications
Architecture Decision Record (ADR)	Selection of integration mechanisms
Interface Design	Responsibilities between modules
Application documentation	Producer-Consumer interactions

Task 4

As CivicConnect transitions into its implementation phase, integration and collaboration will become more crucial towards the success of the project. This section will focus on identifying the appropriate process that will facilitate traceability, verification and reliable integration within the CivicConnect team.

7.1 Software configuration management and Version control:

Software configuration management refers to the process which is used to ensure that all participants of a particular software project follow the same set of guidelines when collaborating. It aims to uphold a consistent standard of quality, integrity, traceability and responsibility for each individual's work (Coursera Staff, 2026).

Version control on the other hand refers to a system used for recording digital changes and modifications of a document. It enables numerous individuals to work on a project simultaneously without interfering with each other's work. By utilising branching to separate individual updates from the main file until they are tested and authorised, version control allows teams to work concurrently. Git functions as a central repository that protects the primary code by creating save points using a commit system (Ibid).

Best practices suitable for a small team:

Branching:

Selecting a git branching strategy is a decision that quietly shapes how a software team releases software. Choosing the wrong strategy opens the team up to 2 main potential risks, namely prolonged merging conflicts and broken or corrupted main branches. The ideal branching strategy that a small team of developers should utilise is the GitHub Flow. The GitHub Flow is the appropriate strategy for small teams, web apps, and any continuous project with a single live production version at a time (F, 2026). The GitHub flow strategy's focal point is the main branch whilst it contains multiple feature branches. It works in the following manner (Rajendran, 2025):

- Create a feature branch, a branch should be created for every modification whether that be a bug fix, a new feature or any improvements.
- Commit changes, team individuals should commit their changes as they progress.
- Open pull-request, once a team member is satisfied with their work a pull-request should be made to merge changes into the main branch.
- Review and test, to ensure quality is maintained the team must engage in code reviews and automated testing.
- Merge and deploy, once the changes have been approved merging and deployment may begin.

Pull requests:

The term pull request also known as PR is used to refer to a method of proposing modifications and changes to a code base (Stryker, 2026). To avoid bottlenecks and confusion, pull requests should be formulated in a manner that is logical, efficient and concise. The following guidelines are the best practices to follow when creating pull requests (Wang, 2026):

- Pull requests should be kept small and focused.
- A code review process with clear standards should be established.
- Tests should be required before merging
- Draft pull requests and continuous integration should be implemented for early feedback.
- Regular code quality checks should be done.
- Reviewers should be assigned based on code ownership and expertise.
- Easy rollback and revert capabilities should be enabled.

- Review feedback should be documented and clear.
- Pull requests must contain clear titles and concise yet informative descriptions.

Peer-reviews:

A key component of high-quality software is the peer review process. This process promotes teamwork, code dependability and improves overall project quality. As a reviewer the main role is to ensure that the code meets project requirements, this includes confirming accuracy, efficiency, security and maintainability (Silva, 2024). Guidelines to peer review effectively:

- Understand the context.
- Check if code adheres to standards.
- Ensure that code is modular and readable.
- Verify that code is functional and correct for its intended purposes.
- Edge cases and potential issues should be considered.
- Feedback should be constructive and actionable.
- Ask questions for clarity where unclear.

Traceability:

Traceability is crucial in software and systems development. Every artefact, requirement, code snippet, and test case are connected by this golden thread which guarantees that the project stays on track and produces the intended results (Hollowell, 2023). The following guidelines should be used to improve traceability (Ibid):

- Clear and consistent artefacts should be established.
- A traceability tool should be implemented.
- Traceability relationships must be defined.
- Traceability should be automated where necessary.
- Traceability links should be regularly reviewed and updated.

CivicConnect potential collaboration risk:

The risk which is likely to occur in the CivicConnect brief is two modifying the same piece of code simultaneously. Should this occur, the fallout could be a number of possible consequences, merge conflicts, inconsistent implementation, broken functionality and loss of time resolving conflicts. The risk can possibly be controlled and contained through the following steps:

- Making use of feature branches instead of direct commits to main.
- Requiring peer reviews from team members before any merging into main can be done.
- Implementing required CI checks.

Incorporating the listed changes above in the team's processes of development will decrease the likelihood of the risk ever occurring.

• Continuous integration and quality gates:

The purpose of continuous integration:

Continuous integration is a fundamental devOps technique, where development teams incorporate code modifications from several contributors into a shared repository. Throughout this process automation is employed for code merging, building and testing. A CI pipeline is the common name for this procedure. When CI is used correctly, companies may find flaws more rapidly and produce better software more efficiently. CI aims to promote quicker and more effective development cycles by resolving major issues and managing code integration difficulties within shared repositories (GitHub, 2026).

Having an automated deployment workflow or simply using git is not synonymous with continuous integration. While CI is a development methodology that automatically integrates and verifies changes through procedures such as building and testing, Git remains a version control system mainly utilised for monitoring and managing changes to source code (GitLab, 2024). CI mostly concentrates on making sure that recently integrated changes are functional and do not cause issues, whereas automated deployment focuses on delivering a validated application to a target environment (F, 2026). Therefore, Git and automated deployment can be used by a software team without ever having implemented effective continuous integration.

CI triggers:

A CI trigger is an automated event setup which specifies the precise start time of a CI/CD workflow. A CI trigger listens for particular activities within a chosen version control system and initiates the automated build and test process instantly, eliminating the need for a developer to manually click “build”, or “run” each time code is written (Belagatti, 2023). Common triggers include:

- Pushing changes to a branch.
- Changes being merged into main branch.
- Updating or opening a pull request.

The right trigger should give developers early feedback while making sure that modifications cannot be added to the protected main branch without first being validated.

For CivicConnect, the opening or updating of a pull request aimed at the protected main branch should initiate CI. This offers automated verification prior to integration, as opposed to finding issues after modifications have been merged. In order to give developers early feedback, pushes to feature branches may also initiate CI. Since the pull-request check directly supports the protected-main and two reviewer workflow, it should also serve as the principal quality gate. Only after passing the necessary automatic checks and receiving the relevant peer approvals can a modification be eligible for merging.

Repeatable build:

A repeatable build is a software development technique that generates software that is functionally equivalent each time it runs by using programmed procedures and saved configurations.

For CivicConnect the following would be required for a repeatable build:

- Database configuration
- Test execution
- Avoiding hard-coded secrets
- Consistent environment configuration
- Build/start commands
- Dependency installation
- Environment variables
- Packaged lock files
- Runtime/version

Appropriate automated checks:

Compilation check:

A compilation check is known as the procedure where a software instrument known as a compiler examines human-readable source code to look for structural or rule-based errors before executing the program (Lenovo, 2026).

For CivicConnect the appropriate steps to ensure a reliable check happens are as follows:

- Installation of lock file dependencies.
- Check that the application launches properly
- Identify any missing packages or invalid imports
- Confirm that project structure and configuration is correct

Unit tests

In software development, a unit test is a technique where the smallest testable components of an application, often referred to as units, are examined separately and independently to ensure proper operation. A unit usually refers to a single class, function, or method (Geek for Geeks, 2026).

For CivicConnect this should include:

- Making a request for services
- Verifying the necessary request data
- Determining whether categories are legitimate or invalid
- Testing changes in status
- Verifying authorisation
- Testing request assignment
- Testing category-specific validation

Static analysis

Static analysis is a software testing technique that looks at the source code, bytecode, or binaries of an application without running the program (DataDog, 2026).

The aim for this analysis is to identify the following issues:

- Problems with syntax and style
- Unused variables
- Possible programming flaws
- Incorrect formatting
- Unreliable code patterns

Dependency/Security check

Dependency changes in pull requests can be found, and vulnerability details can be obtained using GitHub's dependency review feature. Additionally, it can be set up as a necessary check to stop a pull request that introduces vulnerable dependencies from merging.

In the context of CivicConnect it can check the following:

- Updated dependencies
- Recently added dependencies
- Known vulnerabilities
- Potentially problematic dependency versions

Blocking vs warning checks:

Automated check	Treatment	Reason
Compilation check	Blocking	A modification should not be incorporated into main if it cannot be built or started.
Unit test	Blocking	Tests that fail suggest that there may be issues with intended functionality, which should be fixed before integration
Security vulnerabilities	Blocking	It is appropriate to add known vulnerabilities into the shared codebase
Static analysis	Warning initially	Minor style or code quality issues can be fixed during development or review and do not always render the program unusable
Non-critical dependencies	Warning initially	A dependency may have a minor issue that doesn't warrant blocking the integration of an otherwise legitimate modification

Secret and configuration handling:

The CivicConnect repository shouldn't contain sensitive data like authentication tokens, API keys, or database credentials. Rather, they ought to be kept as secure CI secrets and added to the process as needed. Environment variables can be used to handle non-sensitive configuration. Additionally, CI logs should not reveal secrets.

CI result visibility:

In order for reviewers to determine whether the necessary checks were successful or unsuccessful, CI findings ought to be immediately displayed on the pull request. Build, test, and security results should show up as pull request status checks for CivicConnect. This enables reviewers to think about automated verification in addition to their human evaluation, and mandatory checks can stop merging in the event that crucial checks are unsuccessful.

Automation and human peer review:

Automation can reliably and swiftly detect problems like unsuccessful tests, build failures, coding faults, and security flaws, but it is unable to thoroughly assess the context or design of a change. The implementations compliance with requirements, maintainability, adherence to project design decisions and introduction of unforeseen consequences can all be evaluated by human reviewers. As a result, CI should complement the peer review process rather than take its place. Automated checks for CivicConnect offer unbiased confirmation prior to integration, while the necessary human reviewers offer contextual assessment prior to approving the pull request.

Recommended team control:

CivicConnect should utilise a feature branch and pull-request procedure with CI checks included before modifications are merged into the protected main branch. Instead of directly committing to main, developers will work on distinct feature branches and send targeted pull requests.

Every pull request should cause CI to run unit tests, dependency/security checks, static analysis, and compilation verification. While the team can first analyse lower priority warnings, merging should be blocked by build failures, failed tests, and serious security vulnerabilities.

Only once the project's two necessary peer review approvals and the necessary CI checks have been completed will the pull request be merged. In order for reviewers to think about automated verification in addition to their evaluation, CI findings should continue to be accessible on the pull request.

Without requiring a complete CI/CD solution, this method gives CivicConnect traceability, controlled integration, and automated verification. The team's repository rules, continuous integration setup, and associated project documentation should be improved as development moves forward.

Task 5

A2 finding	Evidence/alternative considered	Recommendation	Project decision it should inform	Expected PED/ADR/RTM/application evidence
Design problem 1: status change reactions	Observer vs mediator. One-to-many notifications and audit reactions with reduced coupling supported by the observer	Use observer for autonomous responses to status changes	M2 architecture/design of status-change handling	The pattern is documented by PED/ADR observer-based reactions are displayed in application code
Design problem 2: category specific validation	Factory method vs strategy. Different validation behaviours and extensibility are better supported by strategy.	Use strategy for category specific validation.	M2 validation and application design	Separate validation procedures are displayed by PED/ADR and application code.
Persistence & data	Database centred vs service layer responsibility.	For structural integrity use database constraints along with service layer business rules and transactions	M2 persistence architecture and data integrity decisions	PED/ADR, data model and application transaction/validation logic.
Integration and API	HTTP/REST VS in-process interface: REST offers more freedom and distribution but increases network complexity.	Since modules are part of the same application its best to utilise an in-process interface	M2 architecture and module integration design	PED/ADR, architecture diagram and application interface implementation.
SCM/CI	In addition to direct integration, pull requests, feature branches, peer review, and CI checks were taken into consideration	Use pull requests, protected main, feature branches, 2 approvals, and CI quality gates	Repository rules and collaborative integration process	Repository settings, PR history, CI workflow/results and project documentation.

Purpose of research map:

The research-to-decision map links research results to choices that will be made while CivicConnect is being developed. The recommendations serve as a basis for decisions that should be documented in the pertinent PED, ADR, RTM or application evidence as development advances; they are not regarded as definitive implementation commitments.

Sources

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design patterns: elements of reusable object-oriented software*. Reading, MA: Addison-Wesley.

Martin, R.C. (2003) *Agile software development: principles, patterns, and practices*. Upper Saddle River, NJ: Prentice Hall.

Sommerville, I. (2016) *Software engineering*. 10th edn. Harlow: Pearson Education.

Stevens, W.P., Myers, G.J. and Constantine, L.L. (1974) 'Structured design', *IBM Systems Journal*, 13(2), pp. 115–139.

Guerraoui, R. (1995), Modular Atomic Objects†. *Theory and Practice of Object Systems*, 1: 89-99.
<https://doi.org/10.1002/j.1096-9942.1995.tb00010.x>

Eslamimehr M, Lesani M and Edwards G (2018) 'Efficient detection and validation of atomicity violations in concurrent programs', *Journal of Systems and Software*, 137:618-635, doi:10.1016/j.jss.2017.06.001.

Wang L and Stoller S (2006) 'Runtime analysis of atomicity for multithreaded programs', *IEEE Transactions on Software Engineering*, 32(2):93-110, doi:10.1109/tse.2006.1599419 .

Kistijantoro A, Morgan G, Shrivastava S and Little M (2008) 'Enhancing an Application Server to Support Available Components', *IEEE Transactions on Software Engineering*, 34(4):531-545, doi:10.1109/TSE.2008.38.

Ammann, P., Jajodia, S., & Ray, I. (1996). Ensuring atomicity of multilevel transactions. *Proceedings 1996 IEEE Symposium on Security and Privacy*, 74-84.

Thomas, R. (2000). *Fielding dissertation: CHAPTER 5: Representational state transfer (REST)*. [online] ics.uci.edu. Available at: https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm [Accessed 15 Sept. 2026].